Putting it All Together: ZooKeeper

Gian Pietro Picco
Dipartimento di Ingegneria e Scienza dell'Informazione
University of Trento, Italy
gianpietro.picco@unitn.it
http://disi.unitn.it/~picco

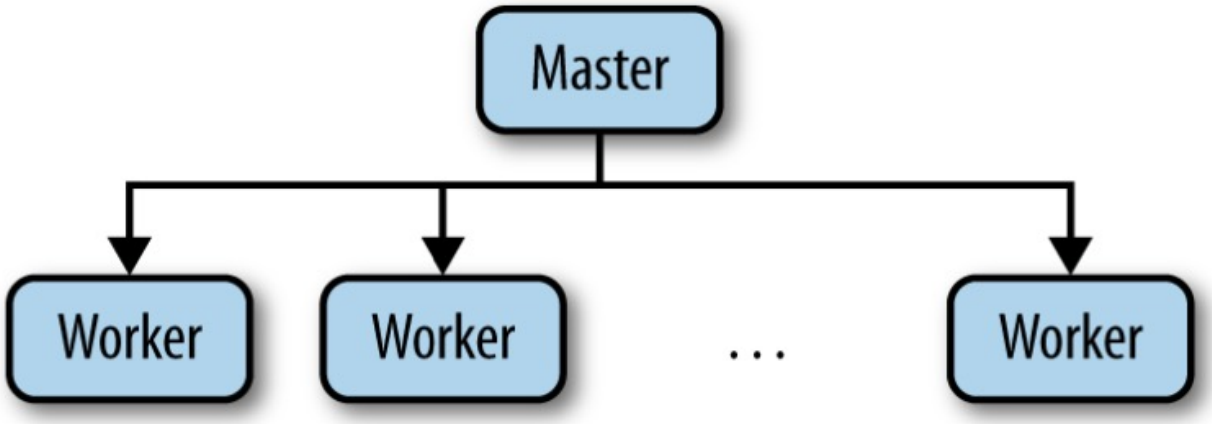
ZooKeeper in a Nutshell

- Modern cloud-based applications run on multiple, frequently changing computers
- Getting distributed systems right is tricky: coordination is crucial
- ***Goal:*** enable developers to focus as much as possible on the application rather than node coordination
- Used by:

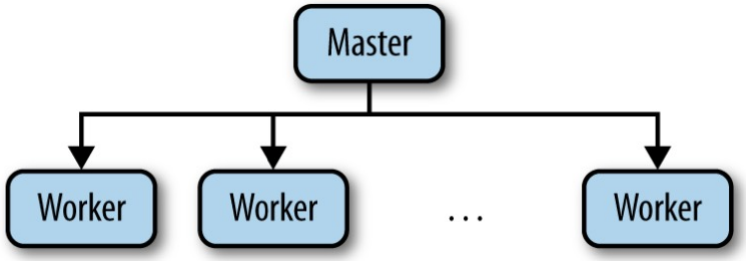
leader election, naming service for service discovery, metadata storage, crash detection and failover, cluster setup and management, …

Example: Master-Worker

- Common architectural pattern in today's distributed systems based on data centers
- Clients:
  - Submit tasks to the master
- Master:
  - Keeps track of pending tasks and available workers
  - Assigns tasks to workers
- Workers:
  - Do the real work



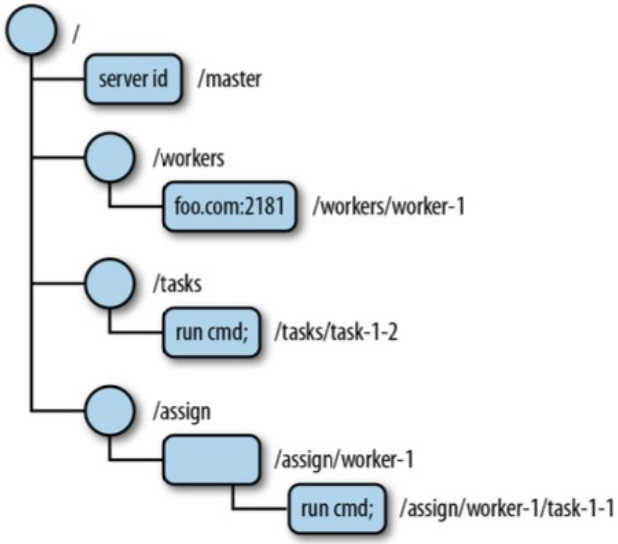
Problems …



- Master crash: can be mitigated with a backup master; however, this backup must
  - Recover the master state somehow
  - Be sure the master is truly non-operational (*"split brain"*)
    - Network partitions may separate the master and the backup…
- Worker crashes
  - All of its non-completed tasks need to be reassigned
    - What is their state? Incomplete? Complete but results not reported? Clean-up operations may be required
  - The master must be able to
    - Detect worker crashes
    - Determine which other workers are available
- Communication failures
  - Workers may become disconnected from the master: could lead to two workers executing the same task
  - Locks may remain in place, blocking other nodes

ZooKeeper Abstraction: Tree-based Namespace

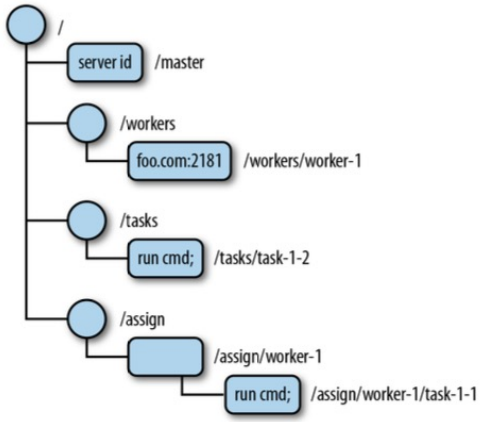
- Similar to a file system
- A tree node is called ***znode*** and can be:
  - ***Persistent:*** can be deleted only explicitly (delete)
  - ***Ephemeral:*** deleted explicitly or automatically when the client that created it is no longer available
    - i.e., disconnected (explicitly or due to faults) or crashed
  - ***Sequential:*** a sequence number is automatically appended to the znode
    - Can be combined with the two above
- An update always overwrites a znode (no partial writes)




```
create  /path data
delete  /path
exists  /path
setData /path data
getData /path
getChildren /path
```

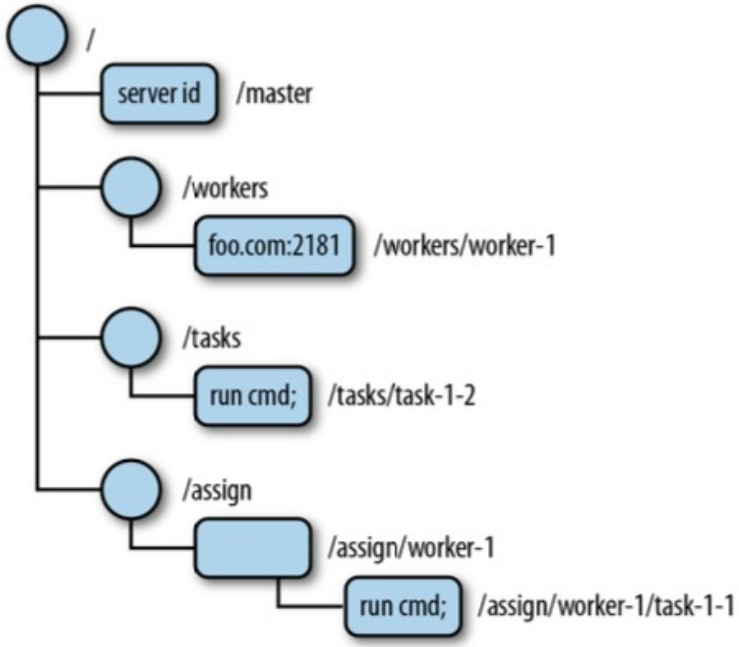
API

Watches



- Blocking operations are not supported: polling the znodes for changes would be inefficient
- Clients can set up a ***watch*** on a znode
  - Asks to be notified when the znode changes
  - The notification is sent only once; it doesn't contain information on the change
  - Provides a base for cache invalidation and management
- Notifications are delivered to a client before any other change is made to same znode
  - Therefore, watching clients see changes to the ZooKeeper state according to a ***global order (sequential consistency)***

Master-Worker

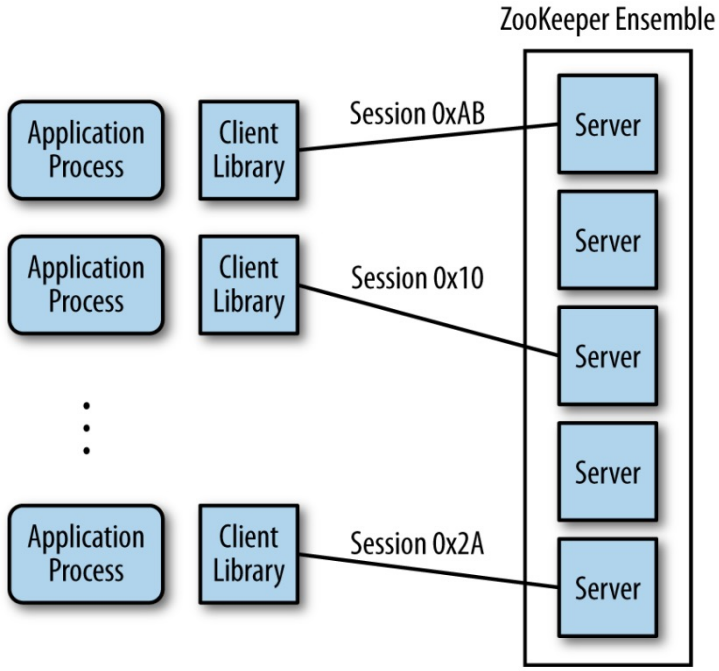


- Only one process is the master:
  - A process creates an ephemeral node `/master`
  - If another node tries to do so, the operation fails: ***mutual exclusion***
  - The other nodes set a watch on `/master` to detect that the znode no longer exist; the backup master will re-create `/master`
- `/workers` is the parent of znodes representing workers availability
  - Each worker adds an ephemeral, sequential node under it
  - The master is notified of availability via a watch on `/workers`
- `/tasks` is the parent of znodes representing pending tasks
  - Clients add tasks under it to request their processing and set a watch to pick up results; the znodes are sequential, effectively providing a queue
  - Watched by the master, to assign tasks
- `/assign` is the parent of all znodes representing assignments of tasks to workers
  - Each worker creates an ephemeral, sequential znode under it, and sets a watch notifying them of the presence of new assigned tasks
  - The master assigns a given task to a given worker by adding a znode under the corresponding folder

Implementing Locks

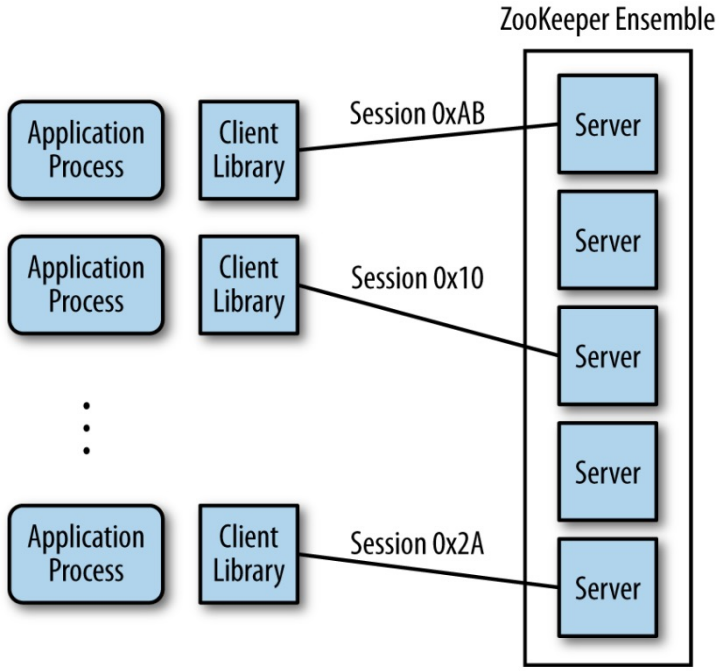
- Simple solution:
  - Acquire lock by creating a znode `/lock`
  - If the lock already exists, this fails; set watch on `/lock` to get notification of deletion and retry
- Problem 1: The process holding the lock crashes
  - Solution: Create `/lock` as an ephemeral znode
- Problem 2: Avoid the "herd effect"
  - Arises with many waiting processes retrying together
  - Solution: Each process
    - Creates a *sequential* znode `/lock/lock-` so that a sequence number is automatically appended to the name
    - Retrieves the list of children of `/lock`;
      - If its sequence number *n* is the smallest, the lock is acquired
      - Otherwise, it watches for the lock with sequence number *n-1*

Architecture



- ZooKeeper servers can operate in ***standalone*** (single server) or ***quorum*** mode
- In quorum mode, a group (***ensemble***) of servers replicates the state (tree) and serves client requests
  - Quorum: minimum number of servers that
    - Must be up and running for ZooKeeper to work
    - Must store the data written by a client before telling it the write operation was successful
  - Read and write operations are carried out via ***quorum-based replication*** ($N_R + N_W > N$, $N_W > N/2$)

Architecture



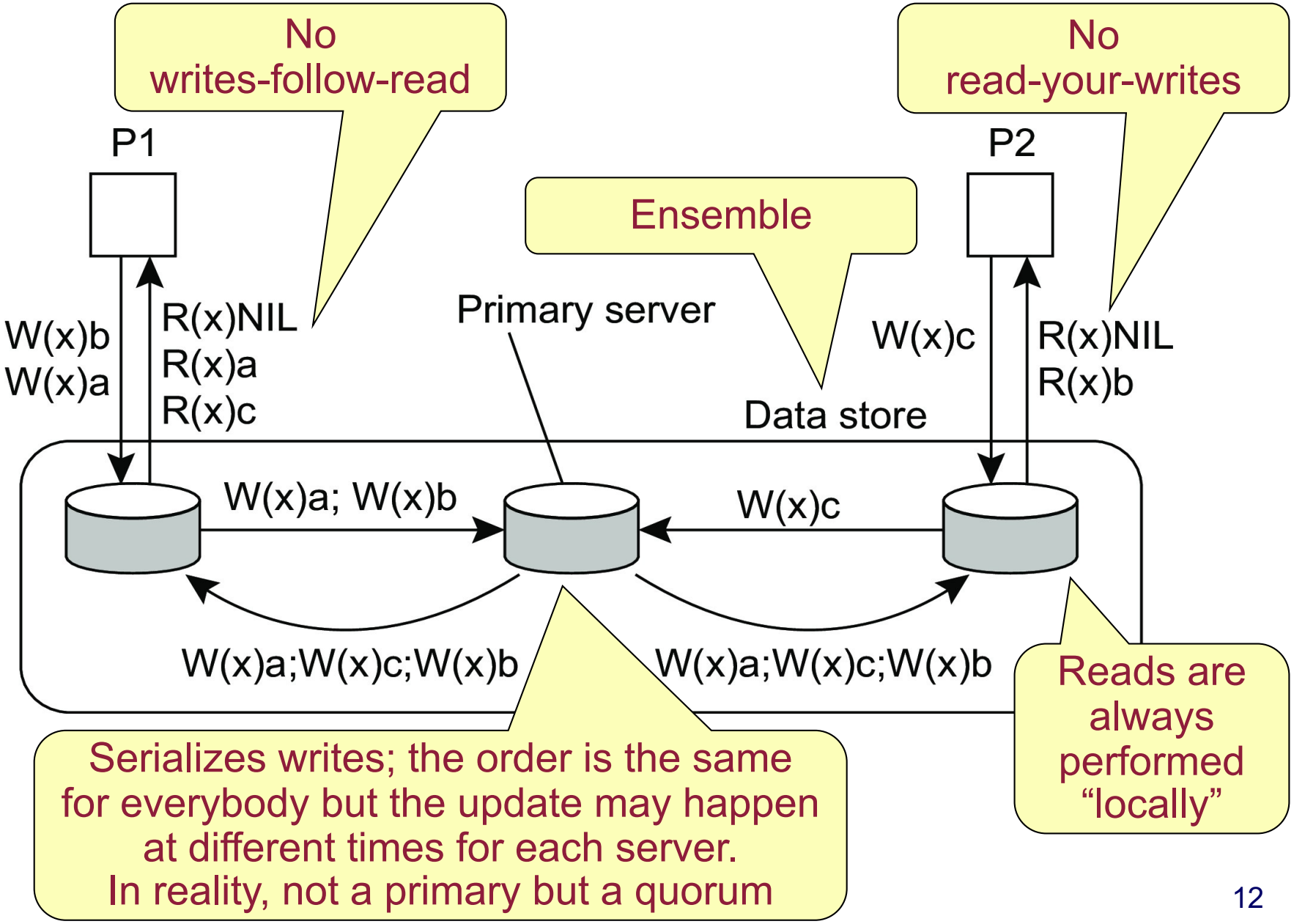
- A client first establishes a session to a single server
  - If the server crashes, ZooKeeper transparently replaces it
  - All client operations are associated with a session
  - A client can have multiple sessions active
- Sessions offer ***FIFO ordering guarantees***
  - Connections are TCP-based…
  - FIFO ordering holds only within a session, i.e.,
    - Not across multiple active sessions
    - Not when a session is closed and reopened, even if they don't overlap in time
- Example (for a given client):
  - Session 1: create **`/tasks`**, create **`/workers`**
  - Session 1 expires (e.g., due to server crash)
  - Session 2: create **`/assign`**

Only **`/tasks`** and **`/assign`** may be created; preserves FIFO in each session, but not across them

Ordering Guarantees

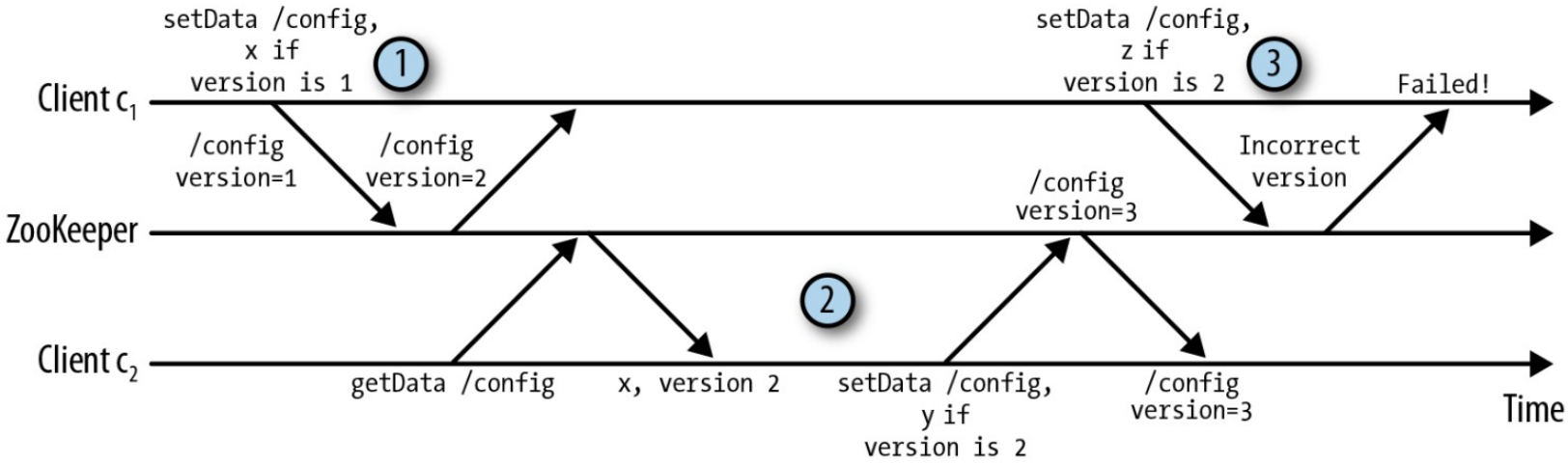
- ZooKeeper's consistency model mixes elements of ***data-centric*** and ***client-centric*** consistency:
  - Writes are ***sequentially consistent***
    - A "primary" (in reality, a quorum) serializes write requests
    - Reads are always performed on the same session server
  - ***Monotonic reads and writes*** are guaranteed
    - Important when switching to another server!
    - Read-your-writes and writes-follow-read are not supported
- The order of writes must be agreed upon by the ensemble
  - However, servers don't need to apply state updates at the same time (and rarely do)
    - What matters to clients is the ordering…
  - Remember that notifications (watches) are also delivered in order and before any other change to the node is performed

Example (simplified…)



Concurrent Writes

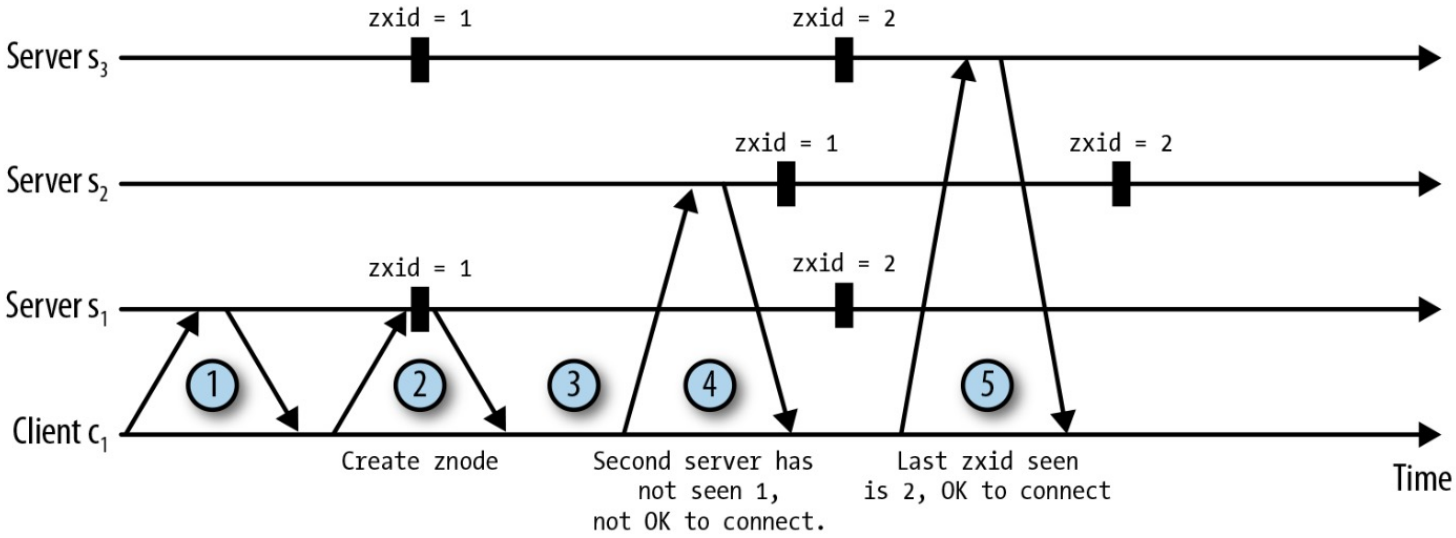
- Concurrent accesses are dealt with ***versions***
  - an operation (`setData`, `delete`) succeeds only if the version number passed by the client matches the one on the server, i.e., it is up-to-date
  - Prevents read-write and write-write conflicts
  - Must be explicitly dealt with by the programmer



1. Client c_1 writes the first version of /config.
2. Client c_2 reads /config and writes the second version.
3. Client c_1 tries to write a change to /config, but the request fails because the version does not match.

Switching Sessions

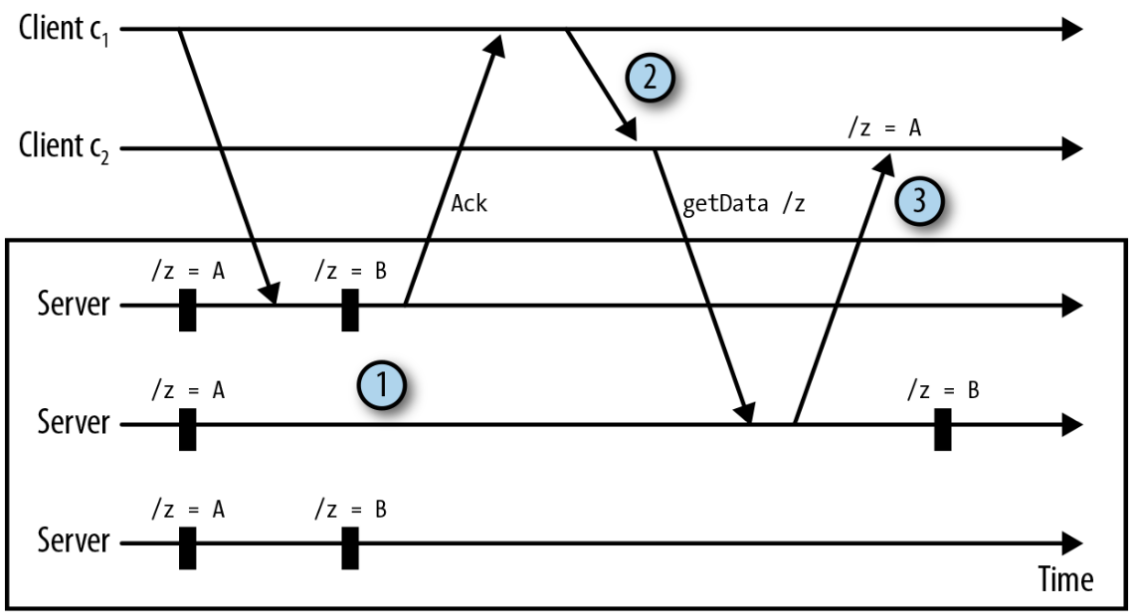
- A client cannot reconnect to an arbitrary server, as it may be behind in processing transactions
- ZooKeeper ensures this is not the case via transaction IDs (*zxid*) preserving ordering guarantees



1. Client connects to Server s_1.
2. Client executes a create operation. The operation succeeds and has zxid 1, assigned by the service.
3. Client disconnects from s_1.
4. Client tries to connect to s_2, but the server has a lower zxid.
5. Client attempt to connect to s_3 succeeds.

Hidden Channels

- ZooKeeper is sometimes used to coordinate clients that interact directly (e.g., via a socket)
- This may break the ordering guarantees



1. ZooKeeper replicates the update across a majority of servers. We omit the messages of the replication protocol to simplify the diagram.
2. c_1 tells c_2 to read /z.
3. c_2 reads a stale (but correctly ordered) value.

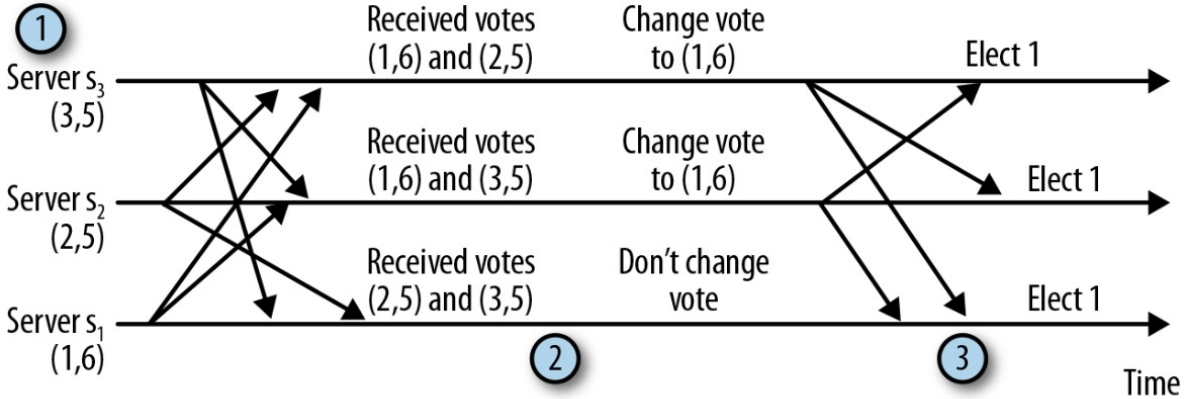
- Addressed by explicitly calling `sync /path` before the read
- The server flushes all pending updates on the target nodes
- Similar to weak consistency model

Leader Election *Inside* ZooKeeper

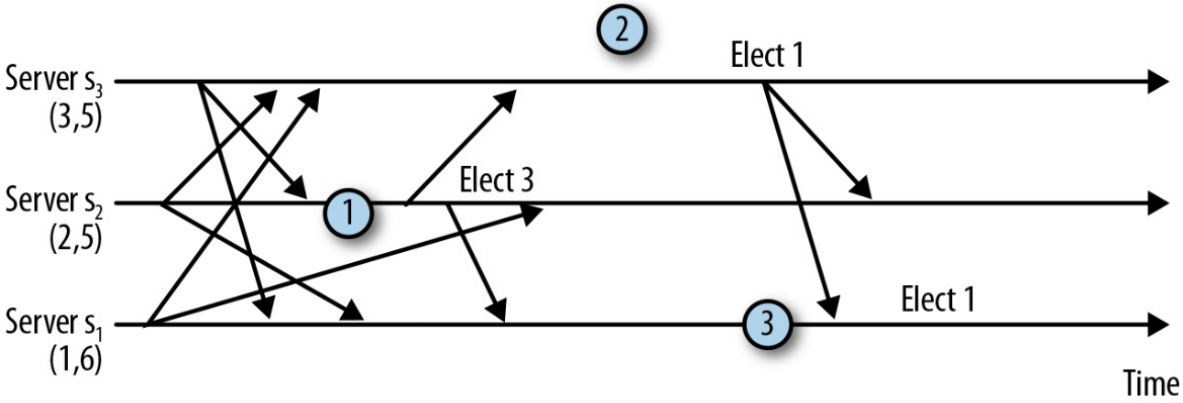
- Each server *s* in the ensemble has an identifier *id(s)* and a monotonically increasing counter *tx(s)* of the latest transaction it handled (i.e., series of operations on the namespace)
- When a follower *s* suspects the leader crashed it:
  - broadcasts an election message with the pair ⟨*voteID, voteTX*⟩. Initially, ⟨*id(s), tx(s)*⟩
  - maintains two variables:
    - *leader(s)*: the server that *s* believes may be the final leader. Initially, is *id(s)*
    - *lastTX(s)*: what *s* knows to be the most recent transaction. Initially, is *tx(s)*
- When *s'* receives ⟨*voteID, voteTX*⟩:
  - If *lastTX(s') < voteTX*
    - *s'* received more up-to-date information on the most recent transaction
    - It sets *leader(s')* ← *voteID*, *lastTX(s')* ← *voteTX*
  - If *lastTX(s') = voteTX AND leader(s') < voteID*
    - *s'* knows as much about the most recent transaction as what it was just sent, but its perspective on which server should be the leader needs to be updated: *leader(s')* ← *voteID*
- When *s'* believes it should be the leader, it broadcasts ⟨*id(s'), tx(s')*⟩
  - Essentially, similar to a ***bullying algorithm:***
    - the most up-to-date server wins; useful in the procedure to restart a quorum
    - if multiple servers are up-to-date the one with the highest ID wins
- Once a server collects the same vote from a quorum, it declares the leader
- All followers connect to the leader and resynch on the latest transaction

Examples

Well-behaved execution



1. Server s_1 starts with vote (1,6), server s_2 with (2,5) and server s_3 with (3,5).
2. Servers s_2 and s_3 change their vote to (1,6) and send a new batch of notifications.
3. All three servers receive the same vote from a quorum and elect server s_1.



1. Server s_2 receives vote (3,5) and changes its vote, forming a quorum. It elects server s_3.
2. Server s_3 receives vote (1,6), but it takes some time to send a new batch of notifications.
3. Server s_1 elects itself leader once it receives the vote of server s_3.

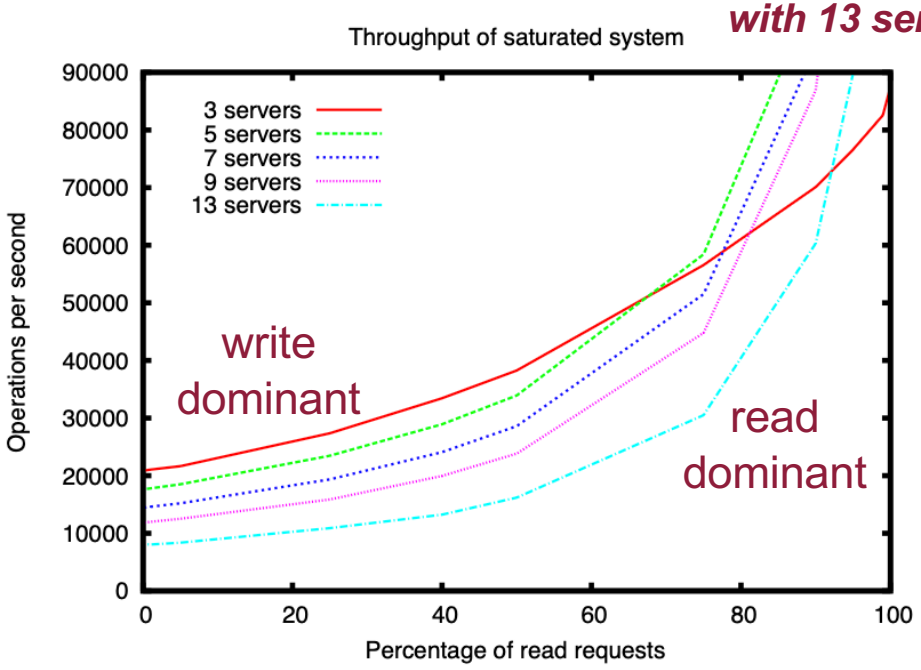
Effect of delays:

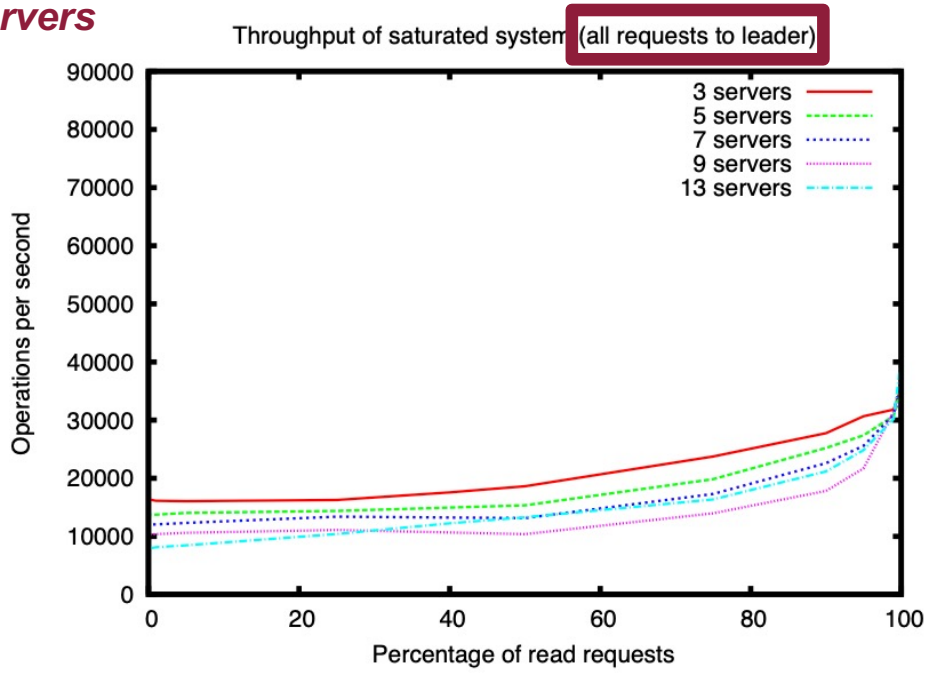
- s_3 will never behave as leader because it knows s_1 is
- However, more time without a leader…
- Mitigated by setting a delay (200ms) before declaring the leader

Performance

- ZooKeeper is designed for workloads where #reads >> #writes ("*from 2:1 to 100:1*")

460k at 100% reads with 13 servers





Many writes: quorum-based replication is expensive; few servers are better

Many reads: multiple servers share the workload, improving performance (and reliability) …

… in comparison with systems (e.g., Google Chubby) sending all request to the leader; performance improves even for many writes

References

ZooKeeper: Wait-free coordination for Internet-scale systems

Patrick Hunt and Mahadev Konar
Yahoo! Grid
{phunt,mahadev}@yahoo-inc.com

Flavio P. Junqueira and Benjamin Reed
Yahoo! Research
{fpj,breed}@yahoo-inc.com

Abstract

In this paper, we describe ZooKeeper, a service for coordinating processes of distributed applications. Since ZooKeeper is part of critical infrastructure, ZooKeeper aims to provide a simple and high performance kernel for building more complex coordination primitives at the client. It incorporates elements from group messaging, shared registers, and distributed lock services in a replicated, centralized service. The interface exposed by ZooKeeper has the wait-free aspects of shared registers with an event-driven mechanism similar to cache invalidations of distributed file systems to provide a simple, yet powerful coordination service.

The ZooKeeper interface enables a high-performance service implementation. In addition to the wait-free property, ZooKeeper provides a per client guarantee of FIFO execution of requests and linearizability for all requests that change the ZooKeeper state. These design decisions enable the implementation of a high performance processing pipeline with read requests being satisfied by local servers. We show for the target workloads, 2:1 to 100:1 read to write ratio, that ZooKeeper can handle tens to hundreds of thousands of transactions per second. This performance allows ZooKeeper to be used extensively by client applications.

1 Introduction

Large-scale distributed applications require different forms of coordination. Configuration is one of the most basic forms of coordination. In its simplest form, configuration is just a list of operational parameters for the system processes, whereas more sophisticated systems have dynamic configuration parameters. Group membership and leader election are also common in distributed systems: often processes need to know which other processes are alive and what those processes are in charge of. Locks constitute a powerful coordination primitive that implement mutually exclusive access to critical resources.

One approach to coordination is to develop services for each of the different coordination needs. For example, Amazon Simple Queue Service [3] focuses specifically on queuing. Other services have been developed specifically for leader election [25] and configuration [27]. Services that implement more powerful primitives can be used to implement less powerful ones. For example, Chubby [6] is a locking service with strong synchronization guarantees. Locks can then be used to implement leader election, group membership, etc.

When designing our coordination service, we moved away from implementing specific primitives on the server side, and instead we opted for exposing an API that enables application developers to implement their own primitives. Such a choice led to the implementation of a *coordination kernel* that enables new primitives without requiring changes to the service core. This approach enables multiple forms of coordination adapted to the requirements of applications, instead of constraining developers to a fixed set of primitives.

When designing the API of ZooKeeper, we moved away from blocking primitives, such as locks. Blocking primitives for a coordination service can cause, among other problems, slow or faulty clients to impact negatively the performance of faster clients. The implementation of the service itself becomes more complicated if processing requests depends on responses and failure detection of other clients. Our system, Zookeeper, hence implements an API that manipulates simple *wait-free* data objects organized hierarchically as in file systems. In fact, the ZooKeeper API resembles the one of any other file system, and looking at just the API signatures, ZooKeeper seems to be Chubby without the lock methods, open, and close. Implementing wait-free data objects, however, differentiates ZooKeeper significantly from systems based on blocking primitives such as locks.

Although the wait-free property is important for per-

Proc. of USENIX Annual Technical Conference, 2010